

Collections: Map, Set & Weak References

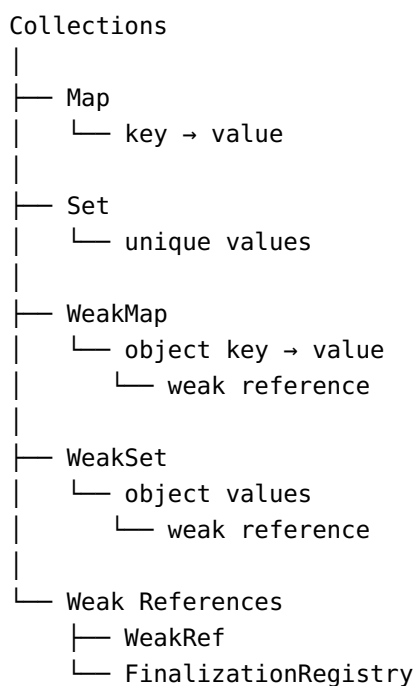
JavaScript-এ data store করার জন্য শুধু `Object` আর `Array` -ই আছে—এমনটা মনে হতে পারে। কিন্তু modern JavaScript-এ আরও কিছু powerful collection আছে:

- `Map`
- `Set`
- `WeakMap`
- `WeakSet`
- `WeakRef`
- `FinalizationRegistry`

এগুলো বিশেষভাবে useful যখন আমাদের দরকার হয়:

- arbitrary keys
- unique values
- fast lookup
- key-value relationships
- object-based metadata
- memory-sensitive caches
- garbage-collection-aware data structures

এই পুরো topic বুঝতে প্রথমে এই mental model:



1. Map কী?

Map হলো JavaScript-এর একটি collection data structure যেখানে **key** → **value** pair store করা হয়।

Basic example:

```
const users = new Map();

users.set("name", "Shariar");
users.set("age", 31);

console.log(users.get("name"));
console.log(users.get("age"));
```

Output:

```
Shariar
31
```

Conceptually:

```
Map
|
├── "name" → "Shariar"
└── "age" → 31
```

2. Map-এর Basic Methods

সবচেয়ে গুরুত্বপূর্ণ methods:

Method	কাজ
<code>set()</code>	key/value add বা update
<code>get()</code>	value retrieve
<code>has()</code>	key আছে কিনা check
<code>delete()</code>	entry remove
<code>clear()</code>	সব entry remove

Method

কাজ

size

কতগুলো entry আছে

Example:

```
const users = new Map();

users.set("name", "Shariar");
users.set("age", 31);

console.log(users.get("name"));
console.log(users.has("age"));

users.delete("age");

console.log(users.has("age"));

console.log(users.size);
```

3. set() Chaining

set() সাধারণত একই Map return করে।

তাই chaining করা যায়:

```
const user = new Map()
  .set("name", "Shariar")
  .set("age", 31)
  .set("role", "Developer");
```

4. Map-এর Key যেকোনো Type হতে পারে

এটা Map-এর সবচেয়ে important feature-এর একটি।

Object-এ property key সাধারণত string বা symbol হয়।

কিন্তু Map-এ key হতে পারে:

- string
- number
- boolean
- object
- array
- function
- symbol
- এমনকি অন্য Map

Example:

```
const map = new Map();

map.set("name", "Shariar");
map.set(100, "One Hundred");
map.set(true, "Yes");

console.log(map.get(100));
console.log(map.get(true));
```

5. Object কে Map Key হিসেবে ব্যবহার

```
const user = {
  id: 1
};

const map = new Map();

map.set(user, "User metadata");

console.log(map.get(user));
```

Output:

```
User metadata
```

এখানে object নিজেই key।

6. Object Key-এর Identity

Map object keys-এর ক্ষেত্রে **reference identity** ব্যবহার করে।

```
const map = new Map();

const user1 = {
  id: 1
};

const user2 = {
  id: 1
};

map.set(user1, "A");

console.log(map.get(user1));
console.log(map.get(user2));
```

Output:

```
A
undefined
```

কারণ:

```
user1 === user2
```

হলো:

```
false
```

দুটোর structure একই হলেও reference আলাদা।

7. NaN এবং Map

Map-এর key comparison-এ **NaN** -কে একই key হিসেবে treat করা হয়।

```
const map = new Map();

map.set(NaN, "Not a Number");

console.log(map.get(NaN));
```

Output:

```
Not a Number
```

8. Map Iteration

Map iterable।

তাই `for...of` ব্যবহার করা যায়।

```
const users = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

for (const entry of users) {
  console.log(entry);
}
```

Output:

```
["name", "Shariar"]
["age", 31]
```

প্রতিটি entry হলো:

```
[key, value]
```

9. Map Destructuring with `for...of`

সবচেয়ে common pattern:

```
const users = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

for (const [key, value] of users) {
  console.log(key, value);
}
```

Output:

```
name Shariar
age 31
```

10. Map-এর Iteration Methods

Map-এর তিনটি গুরুত্বপূর্ণ iterator method:

```
map.keys()
map.values()
map.entries()
```

Example:

```
const map = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

console.log([...map.keys()]);
console.log([...map.values()]);
console.log([...map.entries()]);
```

Output:

```
["name", "age"]  
  
["Shariar", 31]  
  
[  
  ["name", "Shariar"],  
  ["age", 31]  
]
```

11. Map-এর Default Iteration

Map-এর default iterator হলো:

```
map.entries()
```

অর্থাৎ:

```
map[Symbol.iterator]
```

conceptually entries-এর iterator-এর মতো কাজ করে।

তাই:

```
for (const entry of map) {  
  console.log(entry);  
}
```

প্রতিটি entry হিসেবে `[key, value]` পাওয়া যায়।

12. Array থেকে Map

Map constructor iterable entries নিতে পারে।

```
const map = new Map([  
  ["name", "Shariar"],  
  ["age", 31],  
])
```

```
    ["role", "Developer"]  
  ]);
```

এখানে প্রতিটি element:

```
[key, value]
```

format-এ আছে।

13. Map থেকে Array

Spread ব্যবহার করে:

```
const map = new Map([  
  ["name", "Shariar"],  
  ["age", 31]  
]);  
  
const entries = [...map];  
  
console.log(entries);
```

Output:

```
[  
  ["name", "Shariar"],  
  ["age", 31]  
]
```

Keys:

```
const keys = [...map.keys()];
```

Values:

```
const values = [...map.values()];
```

14. Set কী?

Set হলো এমন collection যেখানে **unique values** store করা হয়।

```
const numbers = new Set();

numbers.add(10);
numbers.add(20);
numbers.add(10);

console.log(numbers);
```

10 দুইবার add হলেও একবারই থাকবে।

Conceptually:

```
Set
|
├─ 10
└─ 20
```

15. Set-এর Basic Methods

Method	কাজ
<code>add()</code>	value add
<code>has()</code>	value আছে কিনা check
<code>delete()</code>	value remove
<code>clear()</code>	সব remove
<code>size</code>	number of values

Example:

```
const numbers = new Set([10, 20, 30]);

console.log(numbers.has(20));

numbers.delete(20);
```

```
console.log(numbers.has(20));  
  
console.log(numbers.size);
```

16. Set Automatically Removes Duplicates

```
const numbers = new Set([  
  1,  
  1,  
  2,  
  2,  
  3,  
  3  
]);  
  
console.log(numbers);
```

Conceptually:

```
Set {  
  1,  
  2,  
  3  
}
```

এজন্য array থেকে duplicate remove করতে Set খুব useful:

```
const numbers = [1, 2, 2, 3, 3, 4];  
  
const uniqueNumbers = [...new Set(numbers)];  
  
console.log(uniqueNumbers);
```

Output:

```
[1, 2, 3, 4]
```

17. Set-এর Value যেকোনো Type হতে পারে

```
const set = new Set();

set.add("hello");
set.add(100);
set.add(true);

const user = {
  name: "Sharir"
};

set.add(user);
```

Object-ও value হিসেবে রাখা যায়।

18. Set Object Identity

Object-এর ক্ষেত্রেও reference identity কাজ করে।

```
const set = new Set();

const user1 = {
  id: 1
};

const user2 = {
  id: 1
};

set.add(user1);

console.log(set.has(user1));
console.log(set.has(user2));
```

Output:

```
true
false
```

কারণ দুই object-এর reference আলাদা।

19. Set Iteration

Set iterable:

```
const numbers = new Set([10, 20, 30]);

for (const number of numbers) {
  console.log(number);
}
```

Output:

```
10
20
30
```

Set-এর:

```
set.values()
```

এবং:

```
set.keys()
```

দুটোই values-এর iterator-এর মতো কাজ করে।

এটা একটু surprising:

```
const set = new Set([1, 2, 3]);

console.log(
  set.keys() === set.values()
);
```

এই expression `false` হবে, কারণ দুটি আলাদা iterator object তৈরি হয়।

তবে `keys()` এবং `values()` উভয়ই Set-এর values iterate করে।

20. Set-এর `entries()`

Set-এর `entries()` প্রতিটি value দুইবারের pair হিসেবে দেয়:

```
const set = new Set([10, 20]);  
  
console.log([...set.entries()]);
```

Output:

```
[  
  [10, 10],  
  [20, 20]  
]
```

এটি Map-এর API shape-এর সাথে compatibility রাখার জন্য useful।

21. Map vs Set

Map	Set
Key → Value	Unique Values
<code>set(key, value)</code>	<code>add(value)</code>
<code>get(key)</code>	<code>has(value)</code>
<code>delete(key)</code>	<code>delete(value)</code>
<code>keys()</code>	<code>values()</code>
<code>values()</code>	<code>values()</code>
<code>entries()</code>	<code>entries()</code>
<code>size</code>	<code>size</code>

Mental model:

Map:
userId → user

Set:
userId

```
userId  
userId
```

Set duplicates রাখে না।

22. Map vs Object

এটি খুব important interview topic।

দুটোই key-value data store করতে পারে:

```
const object = {  
  name: "Sharlar"  
};
```

```
const map = new Map([  
  ["name", "Sharlar"]  
]);
```

কিন্তু তাদের উদ্দেশ্য পুরোপুরি এক নয়।

23. Object কখন ব্যবহার করব?

Object ভালো যখন data মূলত একটি **entity/record** represent করে।

Example:

```
const user = {  
  id: 1,  
  name: "Sharlar",  
  age: 31,  
  role: "Developer"  
};
```

এখানে keys fixed-ish domain properties:

```
id  
name
```

```
age  
role
```

এটি একটি user record ।

24. Map কখন ব্যবহার করব?

Map ভালো যখন data structure-এর মূল উদ্দেশ্য:

arbitrary keys → values mapping

Example:

```
const cache = new Map();  
  
cache.set(userObject, expensiveResult);  
cache.set(productId, productData);
```

অথবা dynamic lookup table:

```
const scores = new Map();  
  
scores.set("player1", 100);  
scores.set("player2", 150);
```

25. Map vs Object Comparison

Feature	Map	Object
Key types	Any value	String/Symbol
Size	<code>map.size</code>	আলাদাভাবে বের করতে হয়
Iteration	Built-in	আলাদা methods
<code>for...of</code>	Yes	Directly no
Key/value API	Built-in	<code>Object.keys()</code> etc.
Object keys	Directly supported	String/Symbol conversion
Prototype concerns	সাধারণত নেই	থাকতে পারে

Feature	Map	Object
Semantic purpose	Mapping	Record/entity
Frequent add/delete	Natural API	সম্ভব, কিন্তু Map বেশি natural
Serialization	Built-in JSON support নেই	সহজে JSON হয়

26. Object Key Coercion vs Map Key Identity

এটা খুব important।

Object:

```
const obj = {};

obj[1] = "number";
obj["1"] = "string";

console.log(obj);
```

দুটো একই property key হয়ে যায়:

```
{
  "1": "string"
}
```

কারণ object property key string/symbol হিসেবে handled হয়।

Map:

```
const map = new Map();

map.set(1, "number");
map.set("1", "string");

console.log(map.get(1));
console.log(map.get("1"));
```

Output:

```
number  
string
```

Map number `1` এবং string `"1"` আলাদা key হিসেবে রাখে।

27. Map-এর Key হিসেবে Object

Object-এ:

```
const obj = {};  
  
const key = {  
  id: 1  
};  
  
obj[key] = "data";
```

এখানে object key সাধারণত string conversion-এর দিকে যায়।

Map-এ:

```
const map = new Map();  
  
const key = {  
  id: 1  
};  
  
map.set(key, "data");  
  
console.log(map.get(key));
```

এখানে সত্যিকারের object reference key হিসেবে ব্যবহৃত হয়।

28. Map vs Object — Practical Decision

Object ব্যবহার করো যখন:

আমি একটি entity/record represent করছি।

Example:

```
const product = {  
  id: 101,  
  name: "Laptop",  
  price: 80000  
};
```

Map ব্যবহার করো যখন:

আমার একটি dynamic mapping দরকার।

Example:

```
const productCache = new Map();  
  
productCache.set(productId, product);
```

Easy rule

Object = structured record

Map = dynamic key-value collection

29. Set vs Array

এটাও জানা দরকার।

Array:

```
const numbers = [1, 2, 2, 3];
```

Duplicates রাখতে পারে।

Set:

```
const numbers = new Set([1, 2, 2, 3]);
```

Unique values রাখে।

30. Set কখন ব্যবহার করব?

Set useful যখন দরকার:

- uniqueness
- membership checking
- duplicate removal
- visited items tracking
- selected IDs tracking
- tags/permissions collection

Example:

```
const selectedIds = new Set();

selectedIds.add(10);
selectedIds.add(20);

if (selectedIds.has(10)) {
  console.log("Selected");
}
```

31. WeakMap কী?

WeakMap হলো Map-এর একটি special version যেখানে **key** অবশ্যই **object** বা **non-registered symbol** হতে হবে এবং key-এর reference weakly held হয়।

Basic example:

```
const weakMap = new WeakMap();

const user = {
  name: "Sharlar"
};

weakMap.set(user, {
  lastLogin: Date.now()
});

console.log(weakMap.get(user));
```

32. WeakMap-এর Key

WeakMap-এ সাধারণ primitive key ব্যবহার করা যায় না।

তুল:

```
const weakMap = new WeakMap();  
  
weakMap.set("user", 123);
```

এটি valid WeakMap key নয়।

Object key:

```
const user = {};  
  
weakMap.set(user, 123);
```

valid।

33. WeakMap-এর Basic Methods

WeakMap-এর প্রধান methods:

```
set()  
get()  
has()  
delete()
```

কিন্তু `Map` -এর মতো:

```
weakMap.size
```

নেই।

এবং:

```
weakMap.keys()  
weakMap.values()  
weakMap.entries()
```

নেই।

34. কেন WeakMap Iterable নয়?

এটি intentional।

যদি WeakMap-এর keys iterate করা যেত, তাহলে garbage collection-এর behavior observable হয়ে যেত।

ধরি:

```
const weakMap = new WeakMap();  
  
let user = {};  
  
weakMap.set(user, "metadata");
```

পরে:

```
user = null;
```

এখন object-টির আর অন্য strong reference না থাকলে garbage collector সেটিকে collect করতে পারে।

যদি WeakMap keys iterate করা যেত, তাহলে program theoretically observe করতে পারত কখন key disappear করেছে।

JavaScript সেই ধরনের GC-dependent observability expose করে না।

তাই WeakMap non-iterable।

35. Strong Reference vs Weak Reference

এই distinction খুব important।

Strong reference

```
let user = {  
  name: "Sharlar"  
};
```

`user` variable object-টাকে strongly reference করছে।

যতক্ষণ reachable strong reference আছে, object সাধারণত garbage collection-এর জন্য eligible নয়।

Weak reference

Weak reference object-টাকে alive রাখার জন্য sufficient নয়।

WeakMap এই idea ব্যবহার করে।

36. WeakMap-এর মূল Use Case: Object Metadata

ধরি আমরা কোনো object-এর সাথে extra metadata রাখতে চাই, কিন্তু object-টি destroy হলে metadata-টাও automatically unnecessary হয়ে যাবে।

```
const metadata = new WeakMap();  
  
let user = {  
  name: "Sharlar"  
};  
  
metadata.set(user, {  
  loginCount: 10,  
  lastLogin: Date.now()  
});
```

পরে:

```
user = null;
```

যদি অন্য strong reference না থাকে, user object garbage collection-এর জন্য eligible হতে পারে।

WeakMap-এর associated metadata-ও আর object-টিকে alive রাখে না।

37. WeakMap vs Map

Map	WeakMap
Primitive keys allowed	Object/non-registered symbol keys
Iterable	Non-iterable
<code>.size</code> আছে	<code>.size</code> নেই
keys/values/entries আছে	নেই
Strong key references	Weak key references
General mapping	Object-associated metadata
Cache	Memory-sensitive cache

38. WeakMap-এর Classic Private Data Pattern

Historical JavaScript patterns-এ WeakMap ব্যবহার করে object-এর private-ish state রাখা হতো।

```
const privateData = new WeakMap();

class User {
  constructor(name) {
    privateData.set(this, {
      name
    });
  }

  getName() {
    return privateData.get(this).name;
  }
}
```

ব্যবহার:

```
const user = new User("Sharir");

console.log(user.getName());
```

এখন modern JavaScript-এ native private fields:

```
class User {
  #name;

  constructor(name) {
    this.#name = name;
  }
}
```

থাকার কারণে অনেক ক্ষেত্রে `WeakMap` private storage-এর বদলে `#privateField` বেশি natural।

39. WeakMap দিয়ে Memory-sensitive Cache

Conceptually:

```
const cache = new WeakMap();

function calculate(object) {
  if (cache.has(object)) {
    return cache.get(object);
  }

  const result = expensiveCalculation(object);

  cache.set(object, result);

  return result;
}
```

যদি object-এর আর কোনো strong reference না থাকে, cache entry object-টিকে alive রাখার জন্য বাধ্য করে না।

এটি memory-sensitive caching-এর useful pattern।

40. WeakSet কী?

`WeakSet` হলো Set-এর weak-reference version।

এখানে **object/non-registered symbol values** রাখা যায়।

```
const visited = new WeakSet();
```

```
const user = {};  
  
visited.add(user);  
  
console.log(visited.has(user));
```

Output:

```
true
```

41. WeakSet-এর Methods

প্রধান methods:

```
add()  
has()  
delete()
```

কিন্তু:

```
weakSet.size
```

নেই।

এবং WeakSet iterable নয়।

42. WeakSet কেন Primitive রাখতে পারে না?

```
const weakSet = new WeakSet();  
  
weakSet.add(10);
```

এটি valid নয়।

কারণ WeakSet-এর purpose হলো object identity-এর সাথে weak membership tracking।

43. WeakSet-এর Use Case

ধরি আমরা object-গুলো visited কিনা track করতে চাই:

```
const visited = new WeakSet();

function process(node) {
  if (visited.has(node)) {
    return;
  }

  visited.add(node);

  // process node
}
```

Object পরে আর reachable না থাকলে WeakSet তাকে alive রাখে না।

Graph/tree traversal-এর মতো use case-এ এটি useful হতে পারে।

44. WeakSet vs Set

Set	WeakSet
Primitive allowed	Object/non-registered symbol
Iterable	Non-iterable
<code>.size</code>	নেই
Strong references	Weak references
General uniqueness	Object lifetime-aware membership

45. WeakMap এবং WeakSet-এর Common Idea

দুটোতেই:

```
Object
  ↓
weak reference
```

↓
Collection

Collection object-টাকে alive রাখার জন্য strong reference তৈরি করে না।

এটাই তাদের সবচেয়ে important feature।

46. Garbage Collection কী?

JavaScript-এর memory management-এর একটি গুরুত্বপূর্ণ অংশ হলো **Garbage Collection (GC)**।

যে object program-এর আর কোনো reachable reference-এর মাধ্যমে access করা যায় না, সেটি eventually garbage collection-এর জন্য eligible হতে পারে।

Example:

```
let user = {  
  name: "Sharir"  
};  
  
user = null;
```

এখানে object-টির দিকে আর কোনো known strong reference না থাকলে সেটি garbage collection-এর জন্য eligible হতে পারে।

47. Reachability Mental Model

ধরি:

```
let user = {  
  name: "Sharir"  
};  
  
const anotherReference = user;
```

এখন:

```
user = null;
```

তবুও object alive থাকতে পারে, কারণ:

```
anotherReference
  ↓
Object
```

strong reference আছে।

কিন্তু:

```
anotherReference = null;
```

এর পর যদি আর কোনো strong reference না থাকে, object garbage collection-এর জন্য eligible হতে পারে।

48. Weak Reference কেন দরকার?

কখনো আমরা object-এর সাথে সম্পর্ক রাখতে চাই, কিন্তু সেই relationship যেন object-টাকে memory-তে ধরে না রাখে।

এক্ষেত্রে weak references useful।

```
Strong reference:
Variable —————→ Object
                        ↑
                    keeps alive

Weak reference:
WeakMap - - - - - → Object
                        ↑
                    does not keep alive
```

49. WeakRef

`WeakRef` হলো একটি object reference যা target object-কে strongly keep alive করে না।

Example:

```
let user = {
  name: "Sharir"
};
```

```
const weakRef = new WeakRef(user);
```

এখন target access করতে:

```
const object = weakRef.deref();  
  
console.log(object);
```

যদি object এখনও available থাকে, `deref()` object return করবে।

যদি garbage collected হয়ে থাকে:

```
weakRef.deref();
```

return করতে পারে:

```
undefined
```

50. `WeakRef.deref()`

Basic pattern:

```
const weakRef = new WeakRef(object);  
  
const value = weakRef.deref();  
  
if (value) {  
  console.log(value);  
}
```

কিন্তু একটি গুরুত্বপূর্ণ rule:

`deref()` result ব্যবহার করার সময় object-এর lifetime-এর ওপর assumptions করা যাবে না।

GC কখন object collect করবে সেটা deterministicভাবে control করা যায় না।

51. WeakRef কেন Dangerous হতে পারে?

এই code-এর behavior ধরে নেওয়া ভুল:

```
const object = weakRef.deref();

if (object) {
  // assume object will remain forever
}
```

Weak reference-এর মূল উদ্দেশ্যই হলো target object eventually disappear করতে পারে।

তাই WeakRef ব্যবহার করলে object-এর existence optional ধরে design করতে হবে।

52. WeakRef সাধারণত কখন দরকার?

WeakRef relatively advanced feature।

Potential use cases:

- memory-sensitive caches
- optional object references
- non-essential metadata
- memory-aware libraries
- resource tracking
- avoiding strong retention

কিন্তু normal application logic-এর জন্য WeakRef সাধারণত প্রয়োজন হয় না।

53. FinalizationRegistry

FinalizationRegistry ব্যবহার করে কোনো object garbage-collected হওয়ার পরে একটি cleanup callback schedule করার জন্য registration করা যায়।

Basic example:

```
const registry = new FinalizationRegistry((value) => {
  console.log("Cleaned up:", value);
});
```

```
let user = {  
  name: "Sharar"  
};  
  
registry.register(user, "user metadata");
```

যদি object পরে garbage collection-এর জন্য eligible হয়, registry callback eventually run হতে পারে।

54. `register()` এর ধারণা

Syntax:

```
registry.register(target, heldValue);
```

Example:

```
const registry = new FinalizationRegistry((id) => {  
  console.log(`Object ${id} was cleaned up`);  
});  
  
let user = {};  
  
registry.register(user, "user-123");
```

এখানে:

```
target  
  ↓  
user object  
  
heldValue  
  ↓  
"user-123"
```

55. Finalization Guaranteed নয়

এটা অত্যন্ত important।

FinalizationRegistry callback:

- কখন চলবে জানা যায় না
- সাথে সাথে চলবে না
- নির্দিষ্ট সময়ের মধ্যে চলবে এমন **guarantee** নেই
- application-এর **correctness**-এর জন্য এর ওপর **depend** করা উচিত নয়

তাই:

```
registry.register(...)
```

দিয়ে critical cleanup logic করা উচিত নয়।

56. unregister()

Registration-এর সময় একটি unregister token দেওয়া যায়।

```
const registry = new FinalizationRegistry((value) => {
  console.log(value);
});

let user = {};

const token = {};

registry.register(user, "User data", token);

registry.unregister(token);
```

এতে registration remove করা যায়।

57. WeakRef + FinalizationRegistry Together

দুটো conceptually related:

```
WeakRef
  ↓
weakly reference object

FinalizationRegistry
```

↓
observe eventual cleanup notification

কিন্তু দুটোকে এমনভাবে ব্যবহার করা উচিত নয় যেন JavaScript GC deterministic।

58. WeakRef vs WeakMap

WeakRef	WeakMap
একটি object-এর weak reference	Object → value association
<code>deref()</code>	<code>get()</code>
Iterable নয়	Iterable নয়
Target access করার চেষ্টা করা যায়	Key দিয়ে associated value access
Low-level	Higher-level collection
Advanced use	Metadata/cache-এর common use

59. WeakRef vs FinalizationRegistry

WeakRef	FinalizationRegistry
Object-এর weak reference রাখে	Cleanup notification register করে
<code>deref()</code> দিয়ে target access	Callback eventually run হতে পারে
Object এখনও alive কিনা জানতে চেষ্টা করা যায়	Object collected হওয়ার পর notification
Timing concern	Timing concern আরও বেশি
Optional reference	Cleanup observation

60. WeakMap vs WeakRef

এটা খুব সহজভাবে মনে রাখো:

WeakMap

object → metadata

WeakRef

reference → object

Example:

```
const metadata = new WeakMap();

metadata.set(object, {
  cached: true
});
```

অন্যদিকে:

```
const ref = new WeakRef(object);

ref.deref();
```

61. Memory Leak এবং Collections

Memory leak-এর একটি common pattern:

```
const cache = new Map();

function cacheObject(object, result) {
  cache.set(object, result);
}
```

যদি objects আর application-এ দরকার না থাকে কিন্তু Map-এ strong reference থেকে যায়, তাহলে cache potentially objects-কে alive রাখবে।

Memory-sensitive use case-এ কখনো:

```
const cache = new WeakMap();
```

আরও appropriate হতে পারে।

তবে WeakMap সব cache-এর replacement নয়।

62. WeakMap-এর গুরুত্বপূর্ণ Limitation

এটি মনে রাখা দরকার:

```
const weakMap = new WeakMap();
```

তুমি করতে পারবে:

```
weakMap.set(object, value);  
weakMap.get(object);  
weakMap.has(object);  
weakMap.delete(object);
```

কিন্তু পারবে না:

```
weakMap.size
```

বা:

```
weakMap.keys()
```

কারণ weakly-held entries enumerate করা যায় না।

63. Set Operations — Basic Patterns

JavaScript `Set` দিয়ে mathematical set operations-এর patterns implement করা যায়।

ধরি:

```
const a = new Set([1, 2, 3]);  
const b = new Set([3, 4, 5]);
```

Union

```
const union = new Set([...a, ...b]);  
  
console.log(union);
```

Conceptually:

```
{1, 2, 3, 4, 5}
```

64. Set Intersection

```
const intersection = new Set(
  [...a].filter(value => b.has(value))
);

console.log(intersection);
```

Result:

```
{3}
```

65. Set Difference

```
const difference = new Set(
  [...a].filter(value => !b.has(value))
);

console.log(difference);
```

Result:

```
{1, 2}
```

এই patterns permission, tags, selected IDs, filtering ইত্যাদিতে useful ।

66. Map Frequency Counter

Map-এর practical use case:

```
const words = ["apple", "banana", "apple", "orange", "banana"];

const frequency = new Map();

for (const word of words) {
  frequency.set(
    word,
    (frequency.get(word) ?? 0) + 1
  );
}

console.log(frequency);
```

Conceptually:

```
apple → 2
banana → 2
orange → 1
```

67. Object দিয়ে Frequency Counter বনাম Map

Object দিয়েও করা যায়:

```
const frequency = {};

frequency[word] = (frequency[word] ?? 0) + 1;
```

কিন্তু Map useful হতে পারে যখন keys arbitrary type হতে পারে।

```
const frequency = new Map();

frequency.set(object, count);
```

এটা Object-এর natural use case নয়।

68. Map as Cache

```
const cache = new Map();

function getUser(id) {
  if (cache.has(id)) {
    return cache.get(id);
  }

  const user = fetchUser(id);

  cache.set(id, user);

  return user;
}
```

এখানে Map dynamic lookup/cache-এর জন্য natural choice ।

69. Set as Visited Tracker

Graph traversal:

```
const visited = new Set();

function visit(node) {
  if (visited.has(node)) {
    return;
  }

  visited.add(node);

  // process node
}
```

এটি DSA-তে খুব common pattern ।

70. Map-এর Insertion Order

Map iteration insertion order follow করে ।

```
const map = new Map();

map.set("a", 1);
map.set("b", 2);
map.set("c", 3);

for (const [key, value] of map) {
  console.log(key, value);
}
```

Conceptually:

```
a
b
c
```

Set-ও insertion order অনুযায়ী iterate করে।

```
const set = new Set([3, 1, 2]);

console.log([...set]);
```

Output:

```
[3, 1, 2]
```

Set sorted নয়।

71. Map-এ Existing Key আবার `set()`

```
const map = new Map();

map.set("name", "A");
map.set("name", "B");
```

Map-এ duplicate key entry তৈরি হবে না।

Value update হবে:

```
console.log(map.get("name"));
```

Output:

B

72. Set-এ Existing Value আবার add()

```
const set = new Set();  
  
set.add(10);  
set.add(10);  
set.add(10);  
  
console.log(set.size);
```

Output:

1

73. Map এবং Set-এর Equality

Map/Set-এর key/value uniqueness comparison মূলত **SameValueZero** semantics follow করে।

Practical consequence:

NaN

নিজের সাথে match করে।

```
const set = new Set([NaN]);  
  
console.log(set.has(NaN));
```

Output:

```
true
```

আর `0` এবং `-0` একইভাবে treated হয়।

74. Map/Set এবং JSON

একটি common mistake:

```
const map = new Map([
  ["name", "Shariar"]
]);

console.log(JSON.stringify(map));
```

এটি সাধারণভাবে meaningful Map serialization দেয় না।

JSON-এর native data model-এ Map/Set-এর direct representation নেই।

তাই প্রয়োজন হলে convert করতে হয়।

Map:

```
const object = Object.fromEntries(map);
```

অথবা:

```
const array = [...map];
```

Set:

```
const array = [...set];
```

75. Map → Object

যদি keys strings/symbols-এর উপযোগী হয়:

```
const map = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

const object = Object.fromEntries(map);

console.log(object);
```

Output:

```
{
  name: "Shariar",
  age: 31
}
```

76. Object → Map

```
const object = {
  name: "Shariar",
  age: 31
};

const map = new Map(Object.entries(object));

console.log(map);
```

77. Set → Array

```
const set = new Set([1, 2, 3]);

const array = [...set];
```

অথবা:

```
const array = Array.from(set);
```

78. Array → Set

```
const array = [1, 2, 2, 3];  
  
const set = new Set(array);
```

79. Map vs WeakMap — কখন কোনটা?

Map

যখন:

- entries iterate করতে হবে
- size জানতে হবে
- primitive keys দরকার
- cache manually manage করতে হবে
- general key-value collection দরকার

WeakMap

যখন:

- object-associated data
- object lifetime-এর সাথে metadata tie করতে হবে
- strong retention avoid করতে হবে
- enumeration দরকার নেই

80. Set vs WeakSet — কখন কোনটা?

Set

যখন:

- values iterate করতে হবে
- primitive values রাখতে হবে
- size দরকার
- general uniqueness tracking দরকার

WeakSet

যখন:

- object membership track করতে হবে
- object lifetime-এর সাথে membership tie করতে হবে
- enumeration দরকার নেই
- strong retention avoid করতে হবে

81. Weak References-এর Golden Rule

Weak reference-based APIs ব্যবহার করে কখনো এই assumption করো না:

"Object কখন garbage collected হবে আমি জানি।"

কারণ garbage collection:

non-deterministic

অর্থাৎ কখন GC হবে তার exact timing application control করে না।

82. WeakRef-এর Golden Rule

WeakRef ব্যবহার করে critical application state maintain করা উচিত নয়।

ভুল architecture:

```
Important data
  ↓
WeakRef
  ↓
"Hopefully object থাকবে"
```

ভালো architecture:

```
Critical data
  ↓
Strong reference / proper storage
```

Optional cache/reference



WeakRef

83. FinalizationRegistry-এর Golden Rule

`FinalizationRegistry` cleanup notification-এর জন্য useful হতে পারে, কিন্তু critical cleanup-এর একমাত্র mechanism হওয়া উচিত নয়।

যদি resource cleanup অবশ্যই করতে হয়, explicit cleanup pattern better:

```
try {  
  // use resource  
} finally {  
  // cleanup  
}
```

GC-based finalization-এর timing-এর ওপর critical correctness depend করে না।

84. Map, Set, WeakMap, WeakSet — Complete Comparison

Feature	Map	Set	WeakMap	WeakSet
Stores	Key → Value	Unique Values	Object → Value	Objects
Primitive support	Yes	Yes	No*	No*
Iterable	Yes	Yes	No	No
<code>.size</code>	Yes	Yes	No	No
<code>get()</code>	Yes	No	Yes	No
<code>set()</code>	Yes	No	Yes	No
<code>add()</code>	No	Yes	No	Yes
<code>has()</code>	Yes	Yes	Yes	Yes
<code>delete()</code>	Yes	Yes	Yes	Yes

Feature	Map	Set	WeakMap	WeakSet
<code>keys()</code>	Yes	Yes*	No	No
<code>values()</code>	Yes	Yes	No	No
<code>entries()</code>	Yes	Yes	No	No
Weak references	No	No	Yes	Yes

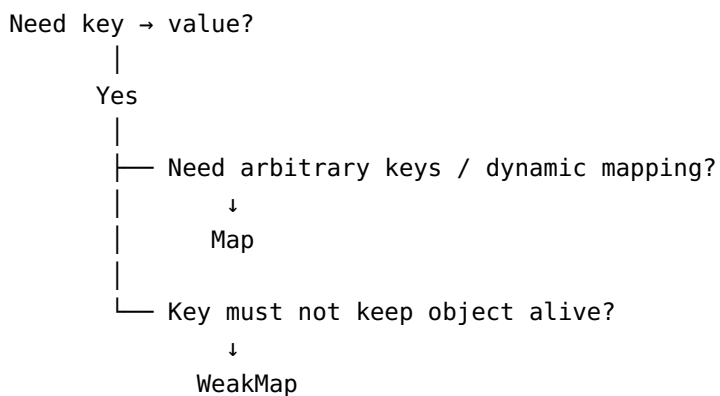
* WeakMap/WeakSet key/value restrictions are based on object/non-registered-symbol support.

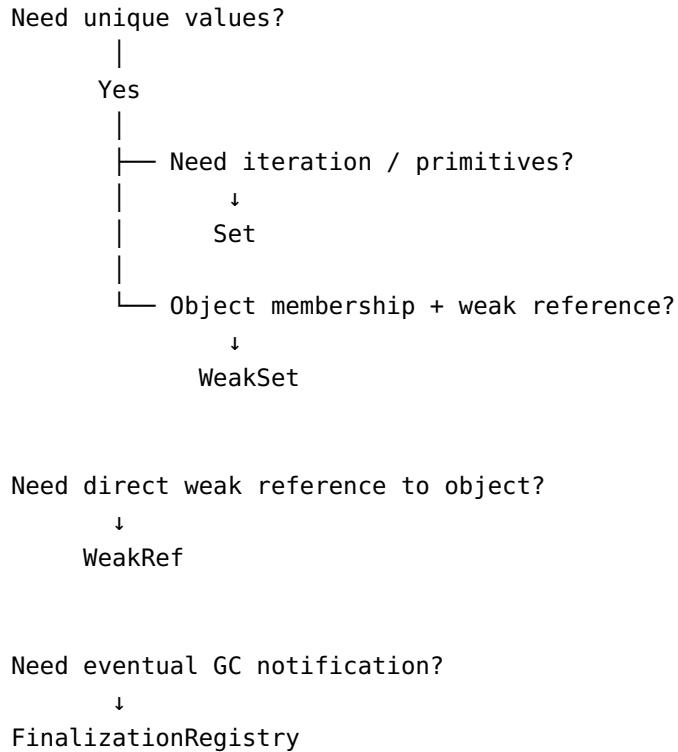
85. WeakRef এবং FinalizationRegistry Comparison

Feature	WeakRef	FinalizationRegistry
Main purpose	Weak reference	GC cleanup notification
Access target	<code>deref()</code>	Direct target access নয়
Depends on GC	Yes	Yes
Timing deterministic	No	No
Common use	Optional reference/cache	Cleanup observation
Beginner-friendly	No	No

86. কোনটা কখন ব্যবহার করব?

একটি simple decision tree:





87. Practical Example — User Metadata

ধরি application-এ user object-এর জন্য UI-specific metadata রাখতে হবে।

```
const metadata = new WeakMap();

function setMetadata(user, data) {
  metadata.set(user, data);
}

function getMetadata(user) {
  return metadata.get(user);
}

const user = {
  id: 1,
  name: "Sharir"
};

setMetadata(user, {
  selected: true
});
```

```
console.log(getMetadata(user));
```

WeakMap এখানে ভালো fit, কারণ metadata-এর lifecycle user object-এর সাথে logically connected ।

88. Practical Example — Duplicate IDs Remove

```
const ids = [
  101,
  102,
  101,
  103,
  102
];

const uniqueIds = [...new Set(ids)];

console.log(uniqueIds);
```

Output:

```
[101, 102, 103]
```

89. Practical Example — Product Lookup

```
const products = new Map([
  [101, { name: "Laptop", price: 80000 }],
  [102, { name: "Mouse", price: 2000 }]
]);

const product = products.get(101);

console.log(product);
```

Map এখানে product ID → product data mapping করছে।

90. Practical Example — Object-based Cache

```
const cache = new WeakMap();

function expensiveOperation(object) {
  if (cache.has(object)) {
    return cache.get(object);
  }

  const result = {
    value: object.value * 2
  };

  cache.set(object, result);

  return result;
}
```

এখানে object-কে cache key হিসেবে ব্যবহার করা হচ্ছে এবং cache object-এর lifetime-কে unnecessarily extend করার জন্য strong key reference দেয় না।

91. Common Mistakes

Mistake 1: Map-এ `map[key]` ব্যবহার করা

ভুল pattern:

```
const map = new Map();

map["name"] = "Shariar";
```

এটি Map-এর actual entry তৈরি করছে না।

সঠিক:

```
map.set("name", "Shariar");
```

Mistake 2: Map-এর জন্য Object API ব্যবহার করা

ভুল:

```
map.name
```

সঠিক:

```
map.get("name");
```

Mistake 3: Set-এ set()

ভুল:

```
set.set(10);
```

সঠিক:

```
set.add(10);
```

Mistake 4: WeakMap-এ primitive key

ভুল:

```
weakMap.set("id", 123);
```

Object/non-registered symbol key ব্যবহার করতে হবে।

Mistake 5: WeakMap-এর .size আশা করা

```
weakMap.size
```

নেই।

Mistake 6: WeakMap iterate করার চেষ্টা

```
for (const entry of weakMap) {  
  // ...  
}
```

কাজ করবে না।

Mistake 7: WeakRef দিয়ে guaranteed lifetime ধরে নেওয়া

```
const object = weakRef.deref();  
  
// "এখন object সবসময় থাকবে"
```

এমন assumption ভুল।

Mistake 8: FinalizationRegistry-কে deterministic cleanup ভাবা

GC কখন হবে তা guaranteed নয়।

92. Interview Questions

1. Map এবং Object-এর main difference কী?

Map হলো general-purpose key-value collection; Object সাধারণত structured record/entity represent করার জন্য বেশি natural।

2. Map-এ object key রাখা যায়?

হ্যাঁ।

```
const map = new Map();  
  
const key = {};  
  
map.set(key, "value");
```

3. Set কেন useful?

Unique values রাখে এবং membership checking-এর জন্য useful ।

4. WeakMap iterable নয় কেন?

কারণ weak references-এর GC behavior enumerate করলে garbage collection-এর behavior observable হয়ে যেত । তাই WeakMap intentionally non-iterable ।

5. WeakMap-এর key কী হতে পারে?

Object বা non-registered symbol ।

6. WeakMap-এর .size নেই কেন?

কারণ garbage collection-এর কারণে বর্তমানে কত entries আছে সেটি deterministicভাবে observable করা design-এর সাথে compatible নয় ।

7. WeakMap কখন ব্যবহার করবে?

Object-associated metadata, memory-sensitive caches এবং object lifetime-এর সাথে tied data-এর জন্য ।

8. WeakSet কী?

Weak reference-based Set, যেখানে object/non-registered symbol membership track করা যায় ।

9. WeakRef কী?

Object-এর weak reference; `deref()` দিয়ে target access করার চেষ্টা করা যায় ।

10. FinalizationRegistry কী?

কোনো registered object garbage-collected হওয়ার পরে eventually cleanup notification পাওয়ার mechanism ।

11. FinalizationRegistry কি deterministic?

না ।

12. Set কি sorted?

না। Set insertion order maintain করে; automatically sorting করে না।

13. Map কি iterable?

হ্যাঁ।

14. WeakMap কি iterable?

না।

93. Output Prediction Practice

Question 1

```
const map = new Map();

map.set(1, "number");
map.set("1", "string");

console.log(map.get(1));
console.log(map.get("1"));
```

Output কী?

Question 2

```
const set = new Set();

set.add(10);
set.add(10);
set.add(20);

console.log(set.size);
```

Question 3

```
const map = new Map([
  ["a", 1],
  ["b", 2]
]);

for (const [key, value] of map) {
  console.log(key, value);
}
```

Output কী?

Question 4

```
const a = {};
const b = {};

const map = new Map();

map.set(a, "A");

console.log(map.get(a));
console.log(map.get(b));
```

কেন দ্বিতীয়টি undefined?

Question 5

```
const set = new Set([1, 2, 2, 3, 3]);

console.log([...set]);
```

Output কী?

94. Practice Tasks

Level 1 — Map Frequency Counter

একটি array:

```
const words = [  
  "apple",  
  "banana",  
  "apple",  
  "orange",  
  "banana",  
  "apple"  
];
```

Map ব্যবহার করে frequency count বের করো।

Expected concept:

```
apple → 3  
banana → 2  
orange → 1
```

Level 2 — Remove Duplicates

```
const numbers = [  
  1, 2, 2, 3, 4, 4, 5, 5  
];
```

Set ব্যবহার করে unique array তৈরি করো।

Level 3 — Object Keys in Map

তিনটি object তৈরি করে Map-এ তাদের metadata রাখো:

```
userObject → role
```

Level 4 — Set Intersection

দুটি Set-এর common values বের করো।

```
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);
```

Expected:

```
{3, 4}
```

Level 5 — WeakMap Metadata

একটি WeakMap বানাও যেখানে user object-এর সাথে:

```
{
  isSelected: true,
  lastVisited: Date.now()
}
```

store করবে।

Level 6 — WeakSet Visited Tracker

একটি function তৈরি করো যা কোনো object আগে process করা হয়েছে কিনা WeakSet দিয়ে track করবে।

Level 7 — Cache Design

ভাবো:

```
calculate(object)
```

expensive computation করে।

একই object-এর জন্য দ্বিতীয়বার calculation না করে cached result return করবে।

এখানে:

Map

নাকি:

WeakMap

কোনটি ব্যবহার করবে এবং কেন?

95. Quick Revision

Map

```
const map = new Map();

map.set("name", "Sharlar");
map.get("name");
map.has("name");
map.delete("name");
map.clear();
map.size;
```

Set

```
const set = new Set();

set.add(10);
set.has(10);
set.delete(10);
set.clear();
set.size;
```

WeakMap

```
const weakMap = new WeakMap();

weakMap.set(object, value);
weakMap.get(object);
weakMap.has(object);
weakMap.delete(object);
```

WeakSet

```
const weakSet = new WeakSet();

weakSet.add(object);
weakSet.has(object);
weakSet.delete(object);
```

WeakRef

```
const ref = new WeakRef(object);

const value = ref.deref();
```

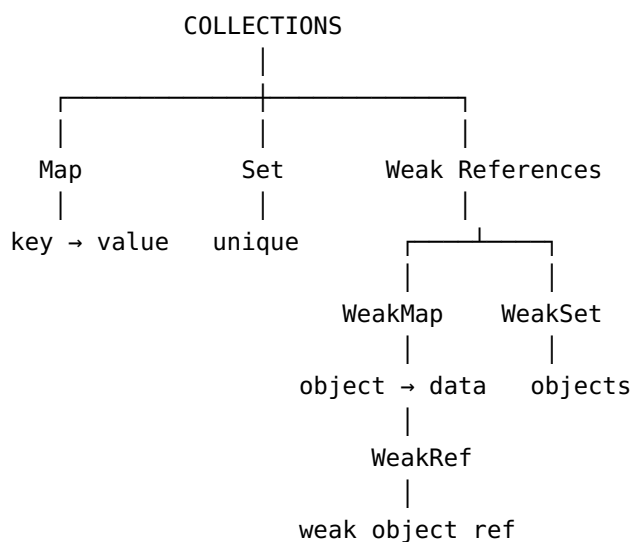
FinalizationRegistry

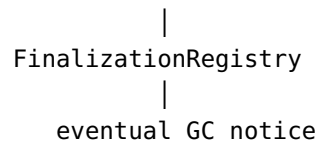
```
const registry = new FinalizationRegistry((value) => {
  // eventual cleanup notification
});

registry.register(object, value);
```

96. Final Mental Model

এই পুরো section-টা এইভাবে মনে রাখো:





আর সবচেয়ে important decision rules:

Structured record?
↓
Object

Dynamic key → value mapping?
↓
Map

Unique values?
↓
Set

Object → metadata, but don't keep object alive?
↓
WeakMap

Track objects without keeping them alive?
↓
WeakSet

Need a direct weak reference?
↓
WeakRef

Need eventual GC notification?
↓
FinalizationRegistry

One-line Memory Trick

Map = mapping

Set = uniqueness

WeakMap = object-এর সাথে data, but weakly

WeakSet = object membership, but weakly

WeakRef = object-এর weak reference

FinalizationRegistry = object cleanup হওয়ার পর eventual notification

সবচেয়ে গুরুত্বপূর্ণভাবে, **Weak APIs**-এর ক্ষেত্রে **garbage collection** কখন হবে সেটা কখনো **deterministic** ধরে নেবে না।